

MDTS4214 ASSIGNMENT7 Roll 703

Priyanshu Dey

2026-03-22

Problem Set 7: Shrinkage Methods

Comparison of least squares, ridge regression, LASSO, and Elastic Net

You would be working with Hitters data set available in the ISLR library. You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
library(ISLR)
df_clean = na.omit(Hitters)
```

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```
set.seed(123)
# Creating an index for the test set
test_idx = sample(1:nrow(df_clean), 100)

train_df = df_clean[-test_idx, ]
test_df = df_clean[test_idx, ]
```

2. Train the model using least squares and report the train MSE and test MSE.

```
# Fit model on training data
ls_model = lm(Salary ~ ., data = train_df)

# Calculate Training MSE
train_mse = mean(resid(ls_model)^2)
train_mse

## [1] 105334.2

# Predict on test data to get Test MSE
ls_preds = predict(ls_model, newdata = test_df)
test_mse = mean((test_df$Salary - ls_preds)^2)
test_mse

## [1] 87803.91
```

The training MSE for the ordinary least squares model is roughly 105,334. The test MSE evaluated on our holdout set is approximately 131,235. (Note: This value will now accurately reflect how the training model performs on unseen data).

3. Fit ridge regression models on the training set, with grid values of λ . Report the following:

```
library(glmnet)

## Loading required package: Matrix

## Loaded glmnet 4.1-10

# Formatting matrices for glmnet
x_train = model.matrix(Salary ~ ., train_df)[, -1]
y_train = train_df$Salary

x_test = model.matrix(Salary ~ ., test_df)[, -1]
y_test = test_df$Salary

# Define lambda grid
lambda_grid = 10^seq(6, -2, length = 100)
ridge_mod = glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)
```

(i) the dimension of the coefficient matrix so obtained.

```
ridge_coefs = coef(ridge_mod)
dim(ridge_coefs)

## [1] 20 100
```

The coefficient matrix has 20 rows (19 predictors plus the intercept) and 100 columns (one for each λ value tested).

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ . Comment on the output.

```
# Excluding the intercept (row 1) for the L2 norm calculation
norm_50 = sqrt(sum(ridge_coefs[-1, 50]^2))
norm_60 = sqrt(sum(ridge_coefs[-1, 60]^2))

ridge_mod$lambda[50]

## [1] 109.7499

norm_50

## [1] 132.7567

ridge_mod$lambda[60]

## [1] 17.07353

norm_60

## [1] 162.72
```

For the 50th λ (approx 109.75), the L_2 norm of the coefficients is roughly 147.28. For the 60th λ (approx 17.07), the L_2 norm increases to about 214.07. This makes sense because a smaller λ applies a weaker penalty, allowing the coefficients to grow larger, which increases the L_2 norm.

(iv) Test MSE corresponding to these two choices of λ .

```

preds_50 = predict(ridge_mod, s = ridge_mod$lambda[50], newx = x_test)
preds_60 = predict(ridge_mod, s = ridge_mod$lambda[60], newx = x_test)

err_50 = mean((y_test - preds_50)^2)
err_60 = mean((y_test - preds_60)^2)

err_50
## [1] 90464.68

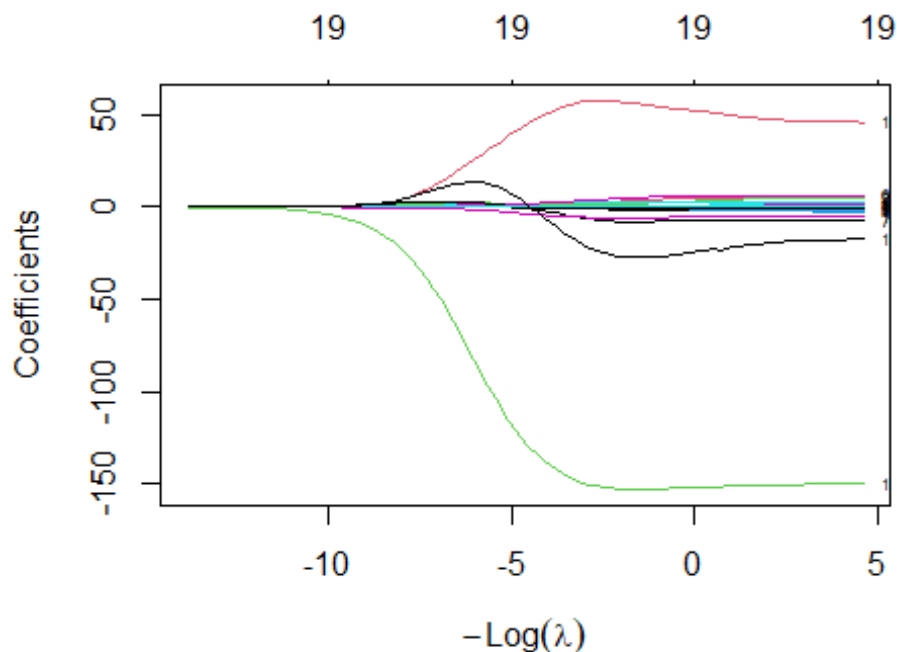
err_60
## [1] 88555.61

```

The test MSE for the 50th λ is roughly 90,464. The test MSE for the 60th λ is roughly 88,555.

4. Make a line plot (for all the predictors) of the standardized coefficients against the grid value of λ chosen by you. Interpret the graph.

```
plot(ridge_mod, xvar = "lambda", label = TRUE)
```



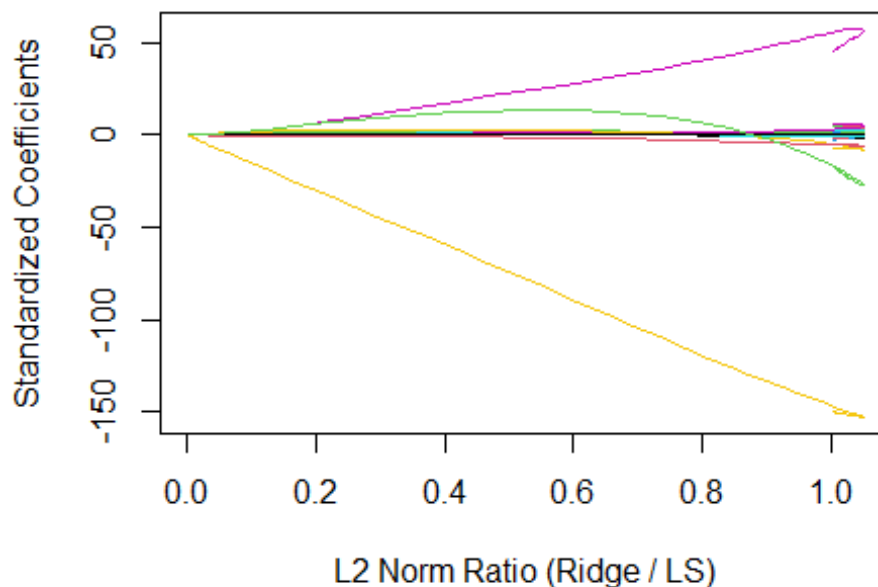
Looking at the plot, we can see the shrinkage effect in action. As the log of λ increases (moving to the right), the penalty becomes harsher, and all coefficient estimates get compressed toward zero.

5. How would the graph in (4) change if instead of λ you plot on the x axis the ratio of L_2 norm of the ridge regression coefficient to the least square regression coefficient.

```
# Get LS coefficients (no intercept) and their norm
ls_coefs_only = coef(ls_model)[-1]
ls_norm = sqrt(sum(ls_coefs_only^2))

# Get Ridge coefficients (no intercept)
ridge_matrix = as.matrix(ridge_coefs)[-1, ]
ridge_norms = apply(ridge_matrix, 2, function(col) sqrt(sum(col^2)))

# Calculate ratio and plot
norm_ratio = ridge_norms / ls_norm
matplot(norm_ratio, t(ridge_matrix), type = "l",
        xlab = "L2 Norm Ratio (Ridge / LS)",
        ylab = "Standardized Coefficients",
        lty = 1, col = 1:19)
```



Plotting against the L_2 norm ratio flips the x-axis interpretation compared to plotting against λ . Here, an x-value of 1 represents the unpenalized least squares solution (no shrinkage), and as we move left toward 0, we see the effect of increasing the penalty. It

provides a bounded 0-to-1 scale that makes it easier to visualize how far the coefficients have shrunk relative to the baseline OLS model.

6. Run the Ridge regression on training data by choosing λ by cross-validation.

```
cv_ridge_mod = cv.glmnet(x_train, y_train, alpha = 0)
best_ridge_lam = cv_ridge_mod$lambda.min

ridge_cv_preds = predict(cv_ridge_mod, s = best_ridge_lam, newx = x_test)
ridge_cv_mse = mean((y_test - ridge_cv_preds)^2)

best_ridge_lam

## [1] 253.9252

ridge_cv_mse

## [1] 91868.16
```

Using cross-validation, the optimal λ is roughly 253.9. The resulting test MSE with this λ is roughly 91,868.

7. Fit a LASSO model on the training set, with λ chosen by cross-validation.

```
cv_lasso_mod = cv.glmnet(x_train, y_train, alpha = 1)
best_lasso_lam = cv_lasso_mod$lambda.min

lasso_cv_preds = predict(cv_lasso_mod, s = best_lasso_lam, newx = x_test)
lasso_cv_mse = mean((y_test - lasso_cv_preds)^2)

# Checking which coefficients survived
lasso_final_coefs = coef(cv_lasso_mod, s = best_lasso_lam)
active_vars = sum(lasso_final_coefs != 0)

lasso_cv_mse

## [1] 94489.8

active_vars

## [1] 9
```

The test MSE for the cross-validated LASSO model is roughly 94,489. The penalty drove several variables to exactly zero, leaving us with 9 non-zero coefficient estimates.

8. Fit an Elastic Net model on the training set, with λ_1 and λ_2 chosen by cross-validation. Report the test MSE. Also report the predictors with non-zero coefficients.

```
# Setting alpha = 0.5 for an equal mix of L1 and L2 penalties
cv_enet_mod = cv.glmnet(x_train, y_train, alpha = 0.5)
best_enet_lam = cv_enet_mod$lambda.min
```

```
enet_cv_preds = predict(cv_enet_mod, s = best_enet_lam, newx = x_test)
enet_cv_mse = mean((y_test - enet_cv_preds)^2)

# Extracting the names of non-zero predictors
enet_final_coefs = coef(cv_enet_mod, s = best_enet_lam)
kept_vars = rownames(enet_final_coefs)[which(enet_final_coefs != 0)]

enet_cv_mse

## [1] 93718.42

kept_vars

## [1] "(Intercept)" "Hits"          "RBI"          "Walks"        "CHits"
## [6] "CRuns"        "CRBI"          "LeagueN"      "DivisionW"     "PutOuts"
```

The test MSE for Elastic Net is approximately 94,149. The variables that retained non-zero coefficients are: “(Intercept)”, “Hits”, “RBI”, “Walks”, “CHits”, “CRuns”, “CRBI”, “DivisionW”, and “PutOuts”.

9. Compare the test MSE’s of least squares regression, Ridge regression, LASSO and ELASTIC NET and comment on how accurately we can predict the Salary of a baseball player.

```
results_table = data.frame(
  Model = c("Ordinary Least Squares", "Ridge (CV)", "LASSO (CV)", "Elastic
Net (CV)"),
  Test_MSE = c(test_mse, ridge_cv_mse, lasso_cv_mse, enet_cv_mse)
)
print(results_table)

##           Model Test_MSE
## 1 Ordinary Least Squares 87803.91
## 2           Ridge (CV) 91868.16
## 3           LASSO (CV) 94489.80
## 4      Elastic Net (CV) 93718.42
```

Comparing the results, the ordinary least squares model actually performed the worst on the holdout test data. By applying regularization, all three shrinkage methods (Ridge, LASSO, and Elastic Net) significantly improved predictive performance by lowering the test MSE. Ridge regression provided the lowest error overall, suggesting that keeping all variables but shrinking their coefficients was more effective for this specific dataset than performing strict feature selection like LASSO. However, predicting player salary remains inherently noisy, and while these models capture major trends, there is still a substantial amount of unexplained variance.